resumer

Développement de logiciels embarqués

USTHB · Ingénieur 5 · Software Engineering · S9

Définitions · Comparaisons · Méthodes · Révision active

Synthèse des trois supports locaux disponibles au 4 novembre 2026. Enseignant : Dr ABADA Lyes. Le cours 2 est rangé dans un fichier « Chapter_2 », tout en affichant encore « Chapter 1 » dans son en-tête : on suit ici les titres Introduction et Architecture ARM. Les autres thèmes du plan général ne disposent pas encore de supports dans le dépôt.

Supports disponibles au 4 novembre 2026

Sommaire

Cours 1 — Systèmes embarqués, architecture et contraintes 3

Cours 1 — Temps réel, acquisition et action 4

Cours 1 — Périphériques et communications 4

Cours 2 — ARM, Cortex et registres 5

Cours 2 — Exécution, interruptions et démarrage 6

Cours 2 — Chaîne de développement et outils 7

Révision active et sources 7

Cours 1 — Systèmes embarqués, architecture et contraintes

Définition et exemples

Un **système embarqué** est un système informatique intégré à un produit pour réaliser une ou plusieurs fonctions déterminées. Il associe matériel, logiciel, alimentation et, selon le besoin, capteurs, actionneurs et interfaces de communication.

Exemples : commande d'un moteur, lave-linge, robot, système de navigation, dispositif de mesure ou objet connecté. Un ordinateur généraliste accueille de nombreuses applications ; un système embarqué est d'abord conçu autour de fonctions et de contraintes du produit. La frontière n'est pas absolue : certains systèmes embarqués exécutent Linux et des applications complexes.

Chaîne fonctionnelle : **mesurer → traiter → décider → agir**, avec communication et stockage éventuels. Tout système embarqué n'est pas connecté à Internet, ni nécessairement soumis à du temps réel dur.

MCU, MPU et SoC

Élément	Caractérisation	Exemple du cours
MCU, microcontrôleur	CPU, mémoires et périphériques largement intégrés	STM32F411
MPU, microprocesseur	Calcul central nécessitant souvent davantage de composants externes	Processeur Cortex-A dans un système complexe
SoC	Intégration de plusieurs fonctions d'un système sur une puce	Calcul, interfaces et accélérateurs selon la puce

La distinction est fonctionnelle et dépend du composant. Le MCU est adapté au contrôle, à l'acquisition et à la basse consommation ; un MPU vise souvent des besoins logiciels et mémoires plus importants.

Contraintes et qualité

Contrainte	Conséquence logicielle
RAM et Flash limitées	Suivre allocations, taille du programme et pile
CPU limité	Maîtriser le temps de traitement
Énergie	Veille, réveils planifiés et communications économes
Taille et coût	Compromis sur composants, interfaces et performances
Fiabilité	Détection d'erreur, surveillance, reprise et état sûr
Délai	Produire le bon résultat dans le temps autorisé

Un **watchdog** déclenche une action de récupération si le logiciel ne le rafraîchit plus comme prévu. Il aide à détecter un blocage ; il ne prouve pas la correction des calculs. Le duty cycling alterne activité et sommeil pour économiser l'énergie.

Cours 1 — Temps réel, acquisition et action

Temps réel dur et souple

En temps réel, la correction dépend du résultat et de son instant de production. « Rapide en moyenne » ne suffit pas à garantir une échéance.

• **Hard real-time** : manquer une échéance constitue une défaillance du système considéré.

• **Soft real-time** : un retard dégrade la qualité sans être nécessairement une défaillance totale.

Exemple du cours : période et échéance de 10 ms, calcul de 3 ms. L'exécution peut respecter ce budget ; une durée de 12 ms dépasse l'échéance. Pour un système réel, il faut aussi compter attente, interruptions et autres tâches : ce simple exemple n'est pas une preuve d'ordonnançabilité générale.

Capteur et actionneur

Un capteur transforme une grandeur physique en signal mesurable ; un actionneur transforme une commande en effet physique. Une boucle de contrôle acquiert la mesure, la compare au besoin et ajuste l'action.

```
Grandeur physique → conditionnement → ADC ou interface numérique
                  → microcontrôleur → traitement → commande → actionneur
```

Un capteur analogique demande souvent une conversion ADC. Un capteur numérique peut convertir en interne puis transmettre par I²C ou SPI. La nature de son interface ne change pas son rôle de mesure.

Échantillonnage

```
fréquence_echantillonnage = 1 / période_echantillonnage
10 ms = 0,010 s → f = 100 Hz → 100 mesures par seconde
```

Augmenter la fréquence augmente les traitements et données à gérer ; choisir selon le phénomène et les contraintes. Le support introduit l'échantillonnage sans développer ici une théorie complète de reconstruction des signaux.

Cours 1 — Périphériques et communications

Périphérique	Fonction
GPIO	Lire ou écrire des signaux numériques
Timer	Mesurer le temps ou produire des événements
ADC	Convertir un signal analogique en valeur numérique
PWM	Produire une commande modulée, par exemple pour luminosité ou moteur
Contrôleur d'interruptions	Organiser les événements et leurs priorités

Comparaison des liaisons

Interface	Principe	Usage / vigilance
UART	Liaison série asynchrone, TX/RX	GPS, console, modules ; paramètres de communication compatibles

Interface	Principe	Usage / vigilance
SPI	Horloge SCK, MOSI, MISO et sélection CS	Capteurs, mémoire, radio ; davantage de fils et sélection des périphériques
I²C	Bus SDA/SCL, appareils adressés	Plusieurs capteurs sur un bus ; vérifier adresses et configuration électrique
CAN	Bus multi-nœud à messages	Automobile et industrie, robustesse et gestion d'erreurs

Les nombres de fils du tableau du cours sont des simplifications : compter aussi la masse et les sélections nécessaires. UART et SPI peuvent être full duplex ; I²C utilise une ligne de données partagée.

Un GPS reçoit des informations de positionnement ; le logiciel lit, valide et extrait les données, par exemple depuis des phrases NMEA. Un module cellulaire permet des échanges sur le réseau mobile. **Se localiser et transmettre une position sont deux fonctions distinctes.**

Logiciel embarqué

Il peut inclure code de démarrage, pilotes, abstraction matérielle, gestionnaires d'interruptions, logique métier et protocoles. Un système bare metal peut se passer d'OS ; un RTOS gère notamment les tâches ; un système plus puissant peut exécuter un Linux embarqué. Toutes les couches présentées dans une architecture ne sont pas obligatoires.

Cours 2 — ARM, Cortex et registres

Du cœur à la carte

Il faut distinguer la conception du cœur ARM, son intégration dans une puce par un fabricant et la carte de développement qui associe cette puce à une alimentation, des connecteurs et d'autres composants. Le STM32F411 intègre un cœur Cortex-M4.

Famille / élément	Rôle principal dans le cours
Cortex-M	Microcontrôleurs, événements, contrôle, bare metal ou RTOS
Cortex-R	Applications temps réel et exigences de fiabilité, dans la classification simplifiée du support
Cortex-A	Applications complexes et OS comme Linux ou Android
ALU	Opérations arithmétiques et logiques
Registres du CPU	Données, adresses, état et contrôle de l'exécution

Ne pas confondre famille de cœur, référence du MCU et nom commercial d'une carte. La liste d'acteurs industriels de la diapositive 7 ne constitue pas une chaîne de fabrication précise de chaque carte.

Registres à mémoriser

Registre	Fonction
R0 à R12	Registres généraux : valeurs, arguments, adresses et résultats
R13 / SP	Pointeur de pile

Registre	Fonction
R14 / LR	Informations de retour d'appel
R15 / PC	Contrôle de la position d'exécution du programme
Registre d'état	Indicateurs et état du processeur

La pile sert notamment aux appels, données temporaires et sauvegardes de contexte. Une pile trop petite peut provoquer une corruption ou une panne. Parmi les indicateurs arithmétiques : zéro, signe négatif, retenue et dépassement ; ils interviennent dans les comparaisons et branchements.

Les registres CPU sont à distinguer des **registres de périphériques**. Par exemple, MODER configure un mode GPIO, IDR lit un état d'entrée et ODR décrit un état de sortie. Les périphériques sont accessibles par des adresses mémoire dédiées : c'est l'accès mémoire mappé aux E/S.

Flash et RAM

La Flash conserve notamment le programme hors tension ; la RAM contient les données d'exécution et perd son contenu à l'arrêt. Certaines zones d'adressage correspondent aux périphériques plutôt qu'à de la RAM. Lire ou écrire une adresse peut donc piloter du matériel.

Cours 2 — Exécution, interruptions et démarrage

Cycle et pipeline

Le cycle simplifié est **fetch → decode → execute**. Un pipeline fait progresser plusieurs instructions à des étapes différentes ; il améliore le débit sans garantir que chaque instruction individuelle soit plus courte. Les adresses incrémentées de quatre octets du support sont illustratives : il ne faut pas en déduire que toute instruction Cortex-M occupe toujours quatre octets.

Polling ou interruption

Polling	Interruption
Le programme vérifie régulièrement une condition	Le matériel signale un événement
Simple à suivre dans une petite boucle	Nécessite gestionnaire, priorités et partage correct des données
Le délai dépend de la fréquence des vérifications	Le délai dépend du masquage, des priorités et du traitement en cours

Déroulement : événement → sauvegarde du contexte nécessaire → gestionnaire ISR → retour à l'exécution. Une ISR courte peut signaler un travail à effectuer ensuite. Une interruption ne supprime pas tout risque de retard.

La **table des vecteurs** contient notamment le SP initial et l'entrée du gestionnaire Reset, puis les entrées d'exceptions et d'interruptions. Au démarrage, le processeur utilise ces informations pour commencer l'exécution correctement.

Événement	Idée essentielle
Reset	Démarrage ou redémarrage
NMI	Interruption non masquable par les mécanismes ordinaires
HardFault	Défaut sérieux du processeur

Événement	Idée essentielle
SysTick	Événement périodique d'un timer système
IRQ périphérique	Événement d'un timer, UART, GPIO, etc.

Les types de faute et détails disponibles dépendent du cœur. Une interruption prioritaire peut préempter une autre selon les règles configurées. Un timer peut déclencher une acquisition toutes les 10 ms, mais le respect de cette période doit être vérifié au niveau système.

Cours 2 — Chaîne de développement et outils

```
Source C → prétraitement → compilation → objets → édition de liens
         → image exécutable → HEX/BIN → programmation Flash → exécution
```

Le compilateur et l'éditeur de liens ont des rôles différents. Les bibliothèques HAL simplifient l'accès aux périphériques ; CMSIS fournit des interfaces standardisées dans l'écosystème Cortex. Le code applicatif peut employer une bibliothèque ou manipuler des registres selon le niveau d'abstraction choisi.

Le débogage utilise traces série, LED, points d'arrêt, observation des variables et registres, analyseur logique ou oscilloscope. **ST-LINK** est une sonde ; **SWD/JTAG** sont des interfaces de communication/debug. Les confondre empêche de comprendre la chaîne outil → sonde → cible.

Le complément de deux pages présente l'ajout du support STM32 dans l'IDE Arduino : installer le paquet de cartes, choisir la carte et vérifier sa présence. Ce n'est pas un chapitre supplémentaire d'architecture. Le lien du document semble avoir perdu des caractères dans sa mise en forme ; utiliser l'URL publiée officiellement par le projet pour une installation effective.

Révision active et sources

1. **Pourquoi un résultat exact peut-il rester incorrect ?** Il arrive après l'échéance d'un système temps réel.

2. **Qui mesure, qui agit ?** Le capteur mesure ; l'actionneur produit l'effet physique.

3. **Que vaut la fréquence pour une période de 5 ms ?** 200 Hz.

4. **SP et PC sont-ils interchangeables ?** Non : pile et exécution sont deux fonctions distinctes.

5. **Une interruption garantit-elle une réaction immédiate ?** Non ; analyser masquage, priorités et latence.

6. **Pourquoi une écriture mémoire peut-elle allumer une LED ?** L'adresse peut désigner un registre GPIO.

7. **Quel intérêt du pipeline ?** Augmenter le débit de traitement d'instructions.

Sources : `COURS/Chapter_1_introduction.pdf` (49 pages), `COURS/Chapter_2_introduction.pdf` (46 pages) et `COURS/Chapter_2_stm32Extension.pdf` (2 pages). Les thèmes Linux embarqué détaillé, synchronisation avancée et méthodes de conception figurent au plan général, sans support dédié présent ; ils ne sont pas inventés dans cette fiche.